

NEXUS ENTERPRISE

SDD + UML DA RECONSTRUÇÃO

React, Docker, Hostinger VPS e Assistente Gemini

Premissa central

O novo sistema será desenhado a partir da operação da empresa. O banco legado será apenas uma fonte de migração, nunca a referência da nova arquitetura.

Versão 2.0 | 13 de julho de 2026

Base analisada: projeto_dados_atual(1).tar

Controle do documento

Campo	Definição
Documento	Software Design Document (SDD) e UML da reconstrução do NEXUS
Interpretação do termo solicitado	"SSD" foi tratado como SDD - Software Design Document
Objetivo	Definir comportamento, arquitetura, experiência, segurança, IA e critérios de qualidade antes da geração do código
Escopo	ERP React, API Python, Docker local, VPS Hostinger, site institucional e integração Gemini
Fora do escopo desta versão	Código-fonte, scripts de migração e arquivos de infraestrutura executáveis
Decisão sobre dados antigos	Serão migrados depois que o novo modelo operacional estiver homologado

Sumário executivo

- Reconstruir o NEXUS como produto web modular, mantendo os fluxos úteis e removendo limitações do Streamlit.
- Criar uma tela de autenticação com card 3D reversível: login na frente e cadastro controlado no verso.
- Hospedar o ERP e um site institucional na mesma VPS Hostinger, isolados por subdomínios e containers.
- Integrar Gemini somente pelo backend, com ferramentas autorizadas, confirmação humana e auditoria.
- Usar a IA para chat, consultas, verificações e criação de lembretes - não para executar ações críticas autonomamente.
- Aplicar padrões contra código prolixo, genérico ou artificial: nenhuma linha sem requisito, nenhuma dependência sem justificativa e revisão humana obrigatória.

Decisão de segurança do cadastro

O efeito de giro não significa cadastro público livre. O verso do card aceitará convite válido ou criará uma solicitação pendente. Perfil e permissões serão definidos por administrador, nunca pelo próprio solicitante.

1. Princípios de reconstrução

1. A operação real vem antes do banco legado.
2. React cuida da experiência; o backend concentra regras, segurança e integrações.
3. O banco novo será criado para o domínio futuro e não para reproduzir tabelas antigas.
4. A IA nunca terá acesso direto ao banco ou a credenciais.
5. Toda ação persistente sugerida pela IA será validada pelo backend e confirmada pelo usuário.
6. Código simples é uma decisão arquitetural: abstrações só existem quando reduzem custo real de manutenção.
7. Ambiente local e VPS usarão as mesmas imagens Docker e os mesmos contratos de serviço.

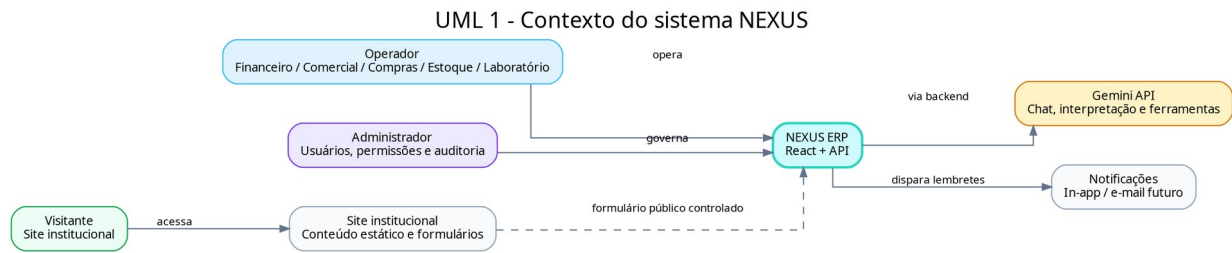


Figura 1 - Usuários, site institucional, ERP, Gemini e notificações.

2. Funcionamento operacional preservado

O novo sistema não copiará telas nem estruturas do legado. Ele preservará os ciclos de trabalho já reconhecidos na operação atual e os reorganizará em jornadas consistentes.

Domínio	Jornada operacional principal	Resultado esperado
Financeiro	Registrar entradas, saídas e transferências; acompanhar vencimentos, fluxo e auditoria.	Visão confiável por unidade, conta, período e situação.
Comercial e patrimônio	Cadastrar clientes/equipamentos, vender ou alugar, movimentar patrimônio e encerrar O.S. comerciais.	Rastreabilidade do ativo e do relacionamento comercial.
Compras	Solicitar, aprovar, comprar, rastrear e receber.	Pedido ligado ao fornecedor e ao recebimento.
Estoque	Cadastrar componente, movimentar, reservar, consumir e inventariar.	Saldo explicável por histórico, local e O.S.
Laboratório	Abrir O.S., diagnosticar, orçar, aprovar, reparar, documentar e entregar.	Ciclo completo com status, fotos, PDFs e responsáveis.
Administração	Gerenciar pessoas, empresas, perfis, permissões e auditoria.	Acesso mínimo necessário e histórico de ações.
Assistente IA	Responder perguntas, verificar pendências e preparar lembretes.	Apoio operacional sem substituir decisões humanas.

3. Experiência base da aplicação

- Menu lateral persistente com módulos permitidos ao usuário.
- Cabeçalho com unidade ativa, busca, notificações, assistente e perfil.
- Rotas reais para que cada tela possa ser atualizada ou compartilhada sem perder contexto.
- Tabelas paginadas no servidor, filtros salvos e estados vazios explicativos.
- Ações críticas em modais ou páginas específicas com resumo antes de confirmar.
- Painel do Gemini recolhível à direita, sem cobrir campos ou tabelas essenciais.
- Responsividade para desktop e tablet; celular prioriza consulta e ações simples.

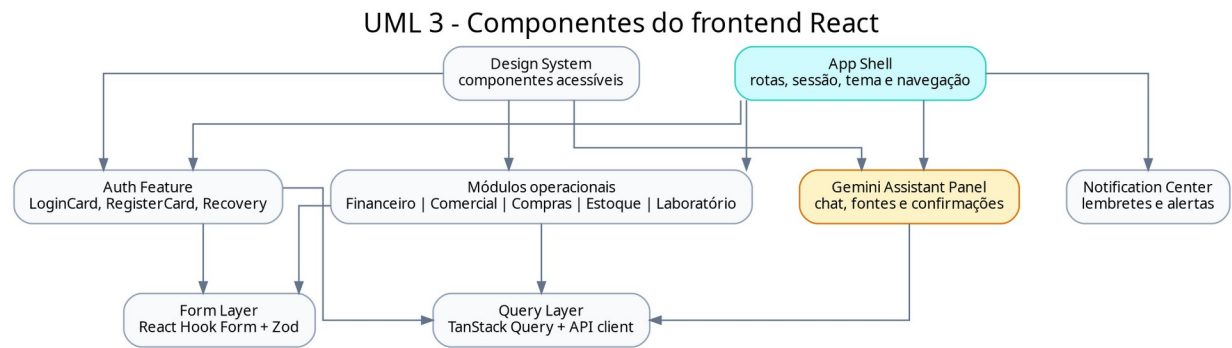
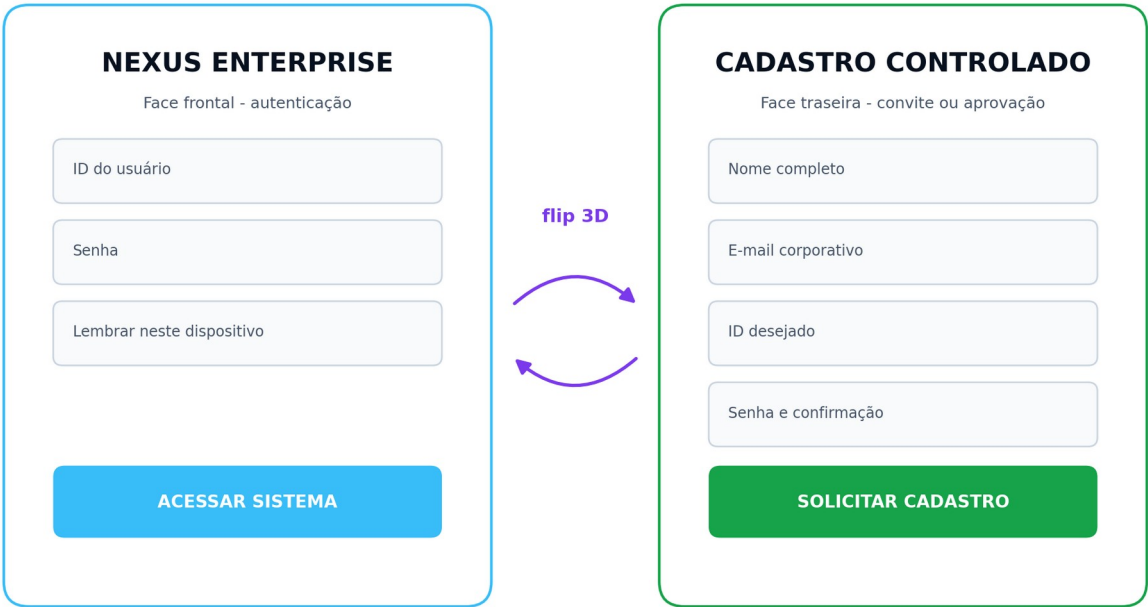


Figura 2 - Componentes do frontend e seus limites.

4. Card dinâmico de login e cadastro

A tela inicial terá fundo institucional e um card central com duas faces. A frente autentica; o verso registra uma solicitação ou consome um convite. O giro é um detalhe de interação, não um atalho de segurança.



A animação é visual; autenticação, convite e aprovação são sempre validados pela API.

Figura 3 - Wireframe funcional do card reversível.

4.1 Requisitos visuais e de interação

ID	Requisito
UX-AUTH-001	Card central responsivo com perspectiva 3D e faces independentes.
UX-AUTH-002	Giro acionado por controle explícito “Cadastrar usuário” ou “Voltar ao login”; cliques em campos não giram o card.
UX-AUTH-003	Transição entre 300 e 450 ms com rotateY e preservação de profundidade.
UX-AUTH-004	Em prefers-reduced-motion, substituir giro por troca instantânea ou fade curto.
UX-AUTH-005	A troca deve funcionar por teclado, leitor de tela e toque.
UX-AUTH-006	Manter foco no primeiro campo da nova face e anunciar a mudança por ARIA live.
UX-AUTH-007	Exibir carregamento, Caps Lock, mostrar/ocultar senha e erros próximos ao campo.

4.2 Campos da face de login

- ID do usuário ou e-mail corporativo.
- Senha.
- Lembrar dispositivo apenas quando política permitir.
- Acessar sistema.

- Recuperar acesso.
- Cadastrar usuário - gira o card.

4.3 Campos da face de cadastro

- Nome completo, e-mail corporativo, ID desejado, senha e confirmação.
- Empresa ou unidade solicitada.
- Código de convite, quando aplicável.
- Aceite de política de uso e privacidade.
- Nenhum campo para escolher perfil ADMIN ou permissões.
- Resultado: conta pendente ou conta ativada por convite pré-autorizado.

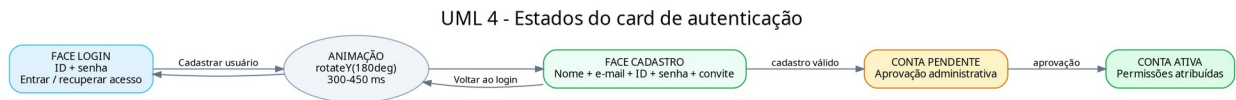


Figura 4 - Estados de interação e ativação do usuário.

5. Autenticação, sessão e autorização

UML 5 - Sequência de login

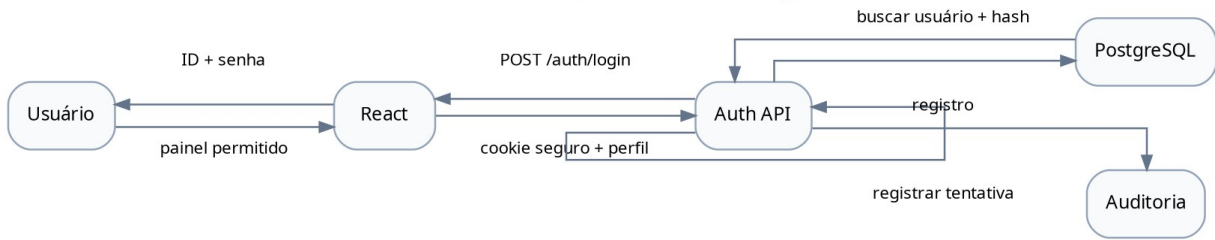


Figura 5 - Login validado no backend; nenhuma senha é tratada pelo frontend além do envio protegido.

UML 6 - Sequência de cadastro controlado

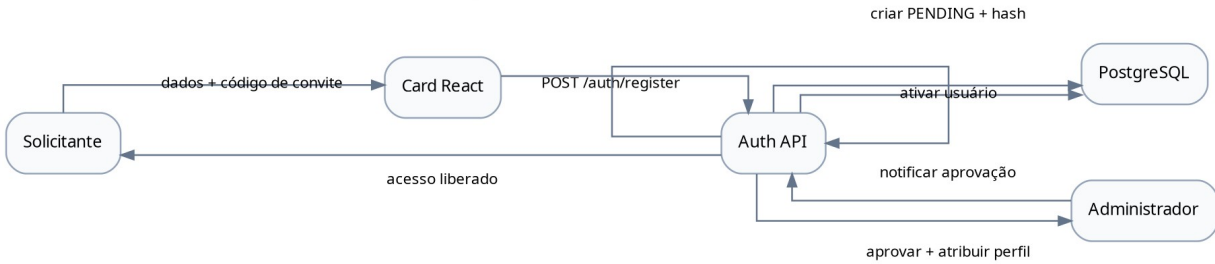


Figura 6 - Cadastro controlado com convite, status pendente e aprovação.

Controle	Regra
Senhas	Hash Argon2id com parâmetros atualizáveis; nunca texto simples.
Sessão	Cookie HttpOnly, Secure e SameSite; token não fica em localStorage.
Bloqueio	Tentativas repetidas geram atraso progressivo e registro de auditoria.
RBAC	Permissão por recurso e ação, com escopo por empresa/unidade.

Controle	Regra
Cadastro	Público irrestrito desativado; convite ou aprovação administrativa.
Recuperação	Token único, curto e revogável, enviado por canal corporativo.
Auditoria	Login, falha, logout, criação, ativação, bloqueio e troca de perfil.

6. Arquitetura técnica

UML 2 - Arquitetura de containers

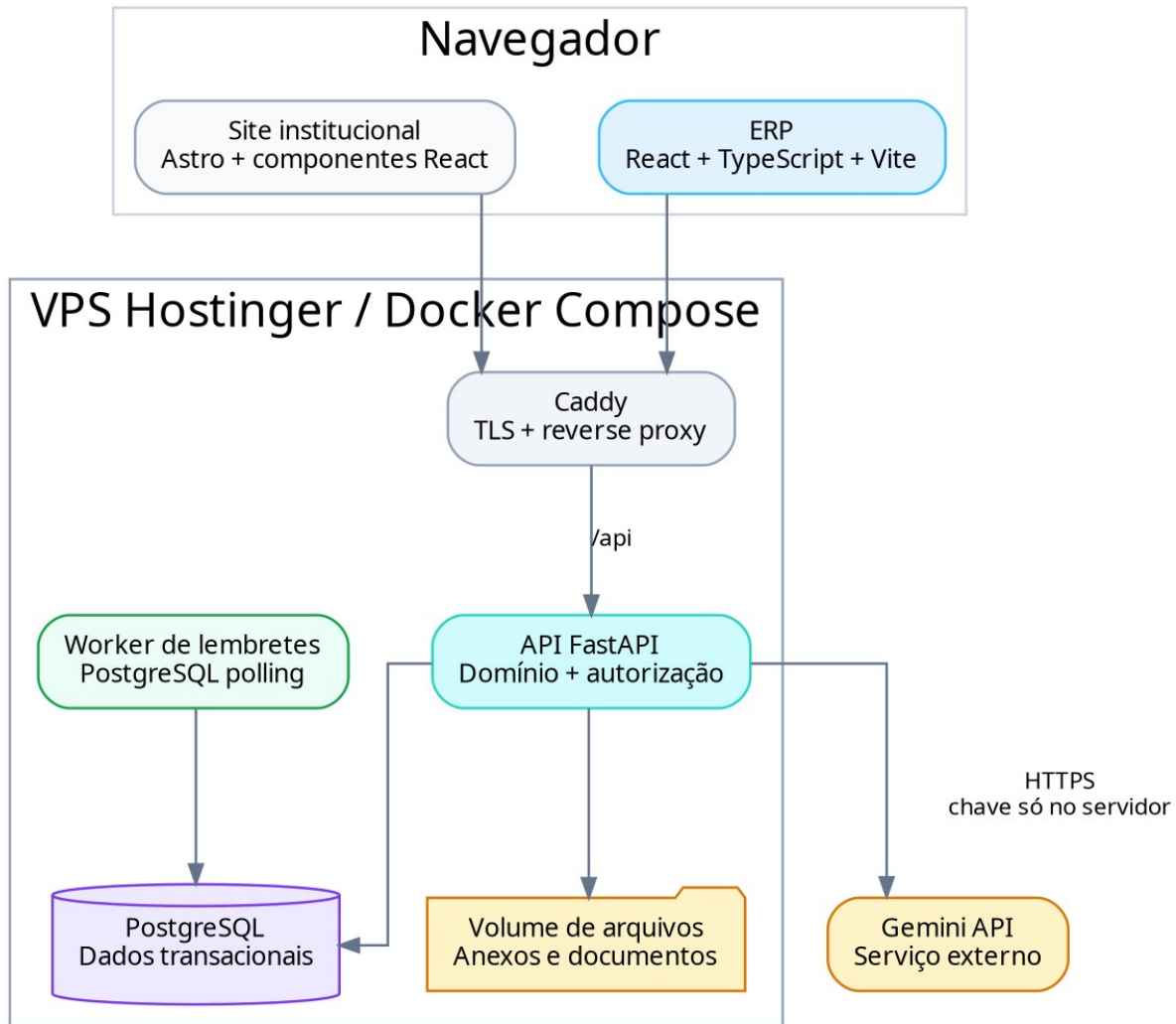


Figura 7 - Containers do ERP, site, API, worker, banco e integração externa.

Arquitetura enxuta

Redis, MinIO, Kubernetes e filas pesadas não entram no primeiro corte sem uma necessidade comprovada. O worker de lembretes usa PostgreSQL com bloqueio concorrente. Componentes opcionais serão adicionados por decisão arquitetural registrada.

6.1 Frontend ERP

- React + TypeScript em modo strict.
- Vite para desenvolvimento e build estático.
- React Router para rotas e proteção de acesso.
- TanStack Query para cache e sincronização com a API.
- React Hook Form + Zod para formulários e validação compartilhável.
- Tailwind CSS e componentes acessíveis próprios; Radix apenas onde resolver comportamento complexo.
- Native fetch encapsulado em um cliente tipado; Axios não é necessário no início.

6.2 Site institucional

- Astro gerando HTML estático, com componentes React apenas onde houver interação.
- SEO, sitemap, metadados sociais, desempenho e formulário protegido contra abuso.
- Servido pelo mesmo Caddy, com domínio principal separado do subdomínio do ERP.
- Nenhum acesso direto ao banco interno; formulários passam por endpoint público isolado.

6.3 Backend

- FastAPI, Pydantic, SQLAlchemy e Alembic.
- PostgreSQL como fonte transacional única.
- Serviços de domínio sem dependência de React ou Streamlit.
- Uploads controlados por tamanho, extensão, MIME e checksum.
- Geração de PDFs e QR Codes em serviço dedicado, ainda no mesmo monólito modular.
- Worker separado para lembretes e tarefas demoradas, reutilizando o mesmo pacote de domínio.

7. Assistente Gemini dentro do NEXUS

O Gemini será um componente auxiliar do produto, não o núcleo das regras. A aplicação enviará somente o contexto mínimo necessário e fornecerá ferramentas tipadas. O modelo poderá decidir qual ferramenta solicitar, mas o backend continuará responsável por autorização, consulta, validação e gravação.

UML 7 - Arquitetura do assistente Gemini

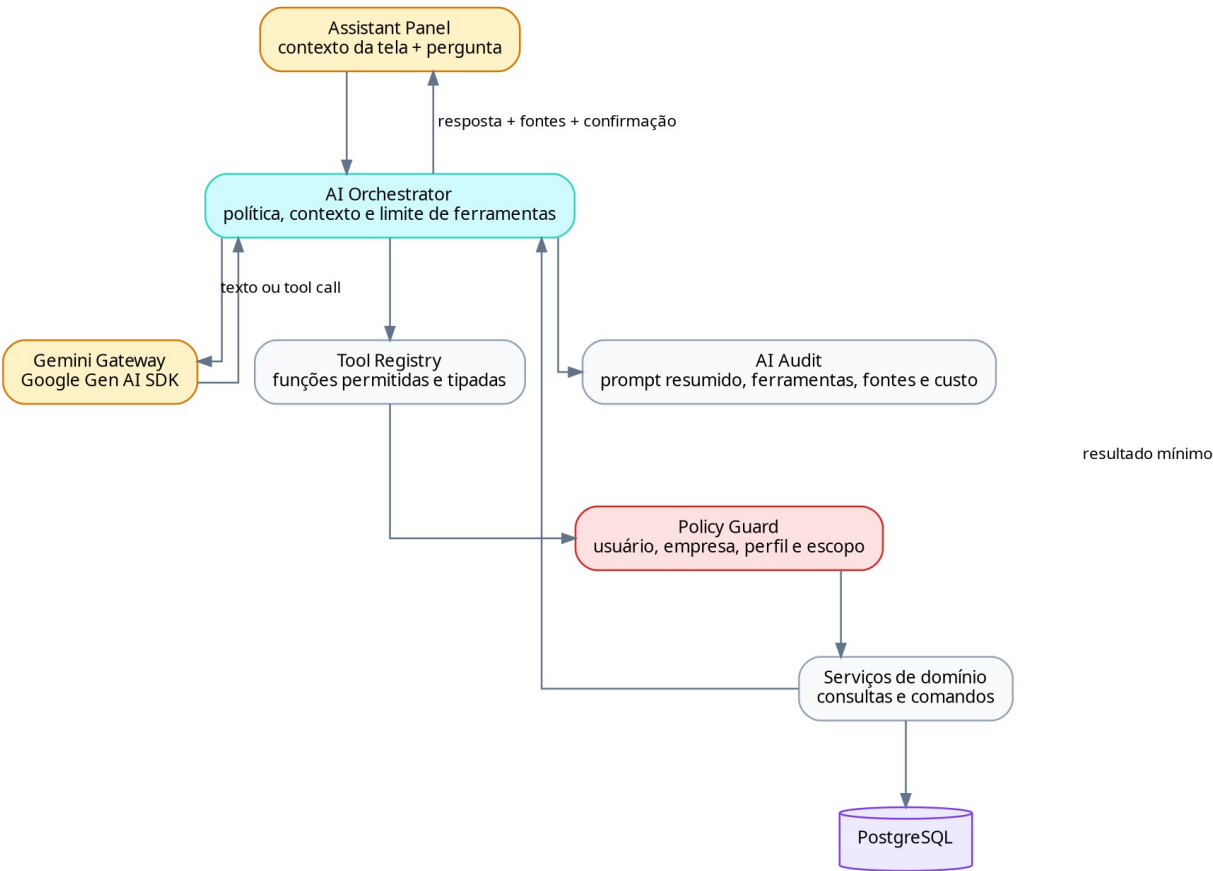


Figura 8 - Orquestrador, gateway Gemini, ferramentas, políticas, serviços e auditoria.

7.1 Casos de uso aprovados para a primeira versão

Categoria	Exemplos	Comportamento
Chat contextual	"Resuma esta O.S.", "Explique estes lançamentos".	Resposta baseada na tela e em consultas autorizadas.
Busca operacional	"Quais O.S. estão atrasadas?", "Quais componentes estão abaixo do mínimo?".	Function calling para ferramentas somente leitura.
Verificação	Detectar ausência de anexo, status parado, vencimento próximo, divergência entre pedido e recebimento.	Resultado com links e identificadores das fontes internas.
Lembretes	"Lembre-me de cobrar o cliente amanhã às 9h".	Cria rascunho estruturado, pede confirmação e persiste.
Redação assistida	Observação, mensagem ao cliente, resumo técnico e minuta de e-mail.	Rascunho editável; nunca envio automático na V1.
Informação externa opcional	Consulta pública atual com grounding, quando habilitado.	Separar claramente informação interna de informação externa e apresentar fontes.

7.2 Ferramentas internas propostas

```
search_customers(query, company_id)
get_work_order(work_order_id)
list_overdue_work_orders(filters)
get_financial_summary(period, company_id)
list_due_receivables(days_ahead)
list_low_stock_items(location_id)
list_delayed_purchase_orders()
create_reminder_draft(text, context)
confirm_reminder(draft_id)
list_user_reminders(status)
cancel_reminder(reminder_id)
```

Cada ferramenta terá schema de entrada e saída, timeout, limite de registros, escopo de empresa, permissão exigida e auditoria. O modelo nunca recebe uma conexão SQL, credencial ou endpoint administrativo.

UML 8 - Consulta operacional com Gemini
contexto mínimo + schema de tools

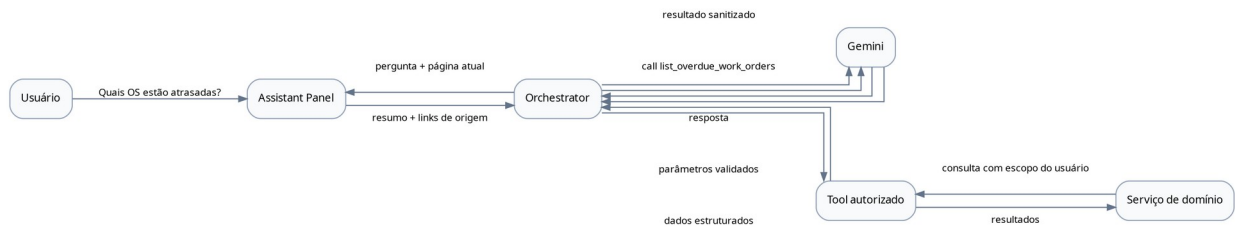


Figura 9 - Pergunta operacional resolvida por function calling e fonte interna.

7.3 Lembretes gerados por IA

UML 9 - Criação de lembrete por linguagem natural
buscar vencidos com lock

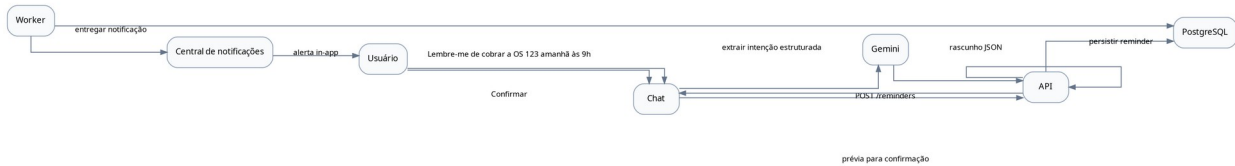


Figura 10 - Extração da intenção, prévia, confirmação, persistência e entrega.

Campo funcional	Regra
Data e hora	Interpretadas no fuso America/Recife e mostradas em prévia absoluta.
Contexto	Pode vincular cliente, O.S., pagamento, compra ou item de estoque.
Entrega inicial	Central de notificações in-app; e-mail e WhatsApp ficam para fase posterior.
Repetição	Somente padrões explícitos; não inferir recorrência silenciosamente.
Confirmação	Obrigatória antes de gravar qualquer lembrete criado pelo chat.
Concorrência	Worker usa bloqueio no banco para impedir entrega duplicada.
Auditoria	Guardar criador, origem IA/manual, confirmação e tentativas de entrega.

8. Segurança, privacidade e limites da IA

UML 12 - Limites de ação da IA

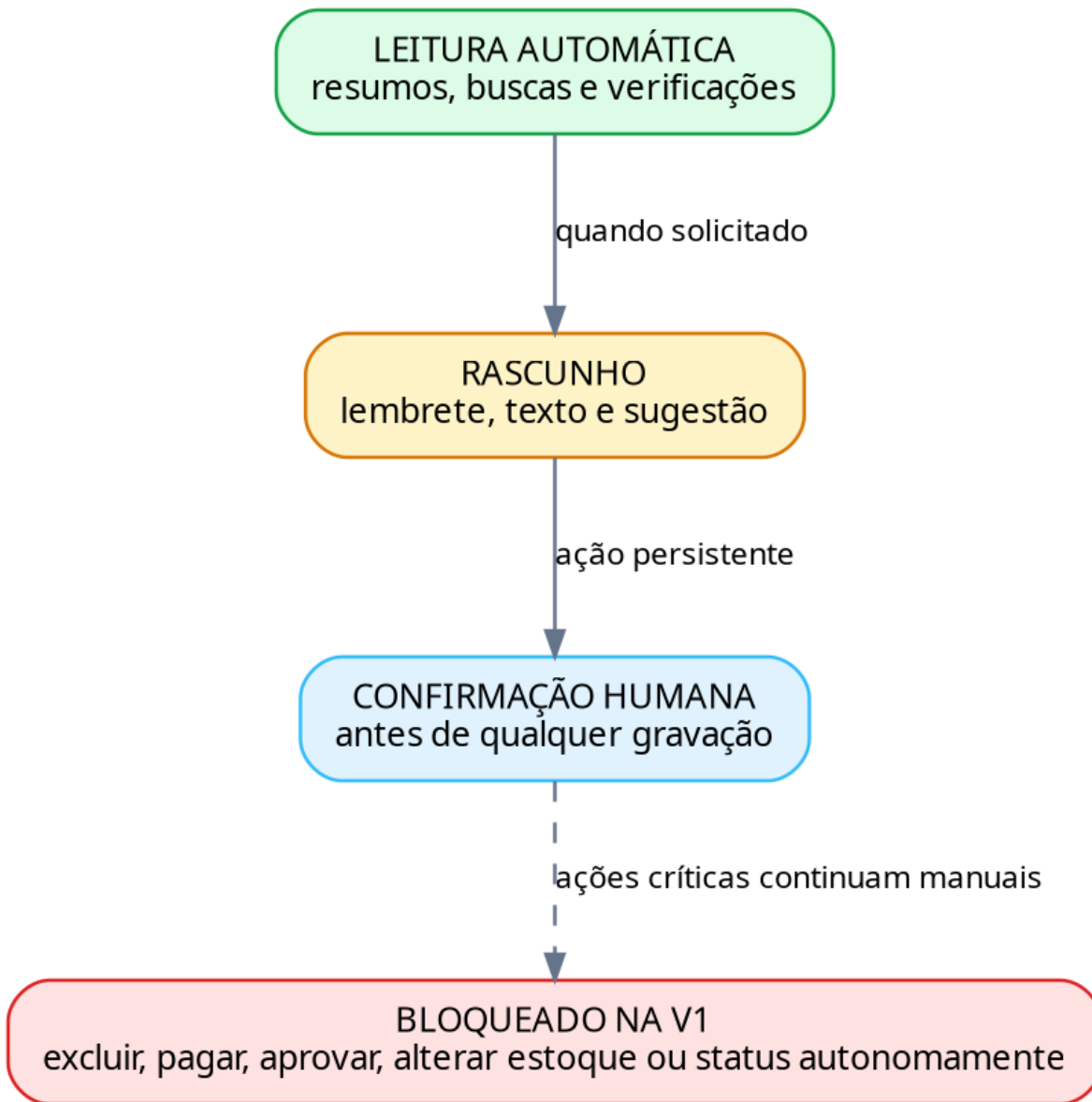


Figura 11 - A IA lê, sugere e prepara; ações críticas continuam humanas.

- A chave Gemini fica somente no backend e em secret de ambiente; nunca no bundle React.
- A chave deve ser restrita à API necessária e rotacionável.
- Plano pago deve ser usado em produção quando houver dados corporativos, seguindo a política contratual escolhida.
- Enviar apenas campos necessários; remover senhas, chaves, tokens e documentos completos quando um recorte for suficiente.
- Não enviar um banco inteiro, dumps ou listas sem limite.
- Definir limite de tokens, tempo, custo por usuário e máximo de chamadas de ferramenta por interação.

- Registrar fonte interna, ferramenta chamada, duração, resultado e confirmação; evitar salvar conteúdo sensível integral quando um hash ou resumo for suficiente.
- Proteger contra prompt injection em anexos e textos de usuários: conteúdo recuperado é dado, não instrução de sistema.
- Bloquear na primeira versão: exclusão, pagamento, aprovação financeira, alteração de estoque e mudança de status autônoma.

Seleção de modelo

O nome do modelo será variável de ambiente. Como referência técnica em 13/07/2026, a documentação oficial apresenta gemini-3.5-flash e recomenda a Interactions API para os recursos mais recentes. A arquitetura não ficará presa a um modelo específico.

9. Padrão de código enxuto e funcional

A exigência “não deixar o código com cara de IA” será tratada como um padrão objetivo de engenharia: código com vocabulário do domínio, sem comentários óbvios, sem abstrações genéricas, sem arquivos inflados e sem funcionalidades especulativas. Código assistido por IA recebe a mesma revisão e os mesmos testes que qualquer implementação humana.

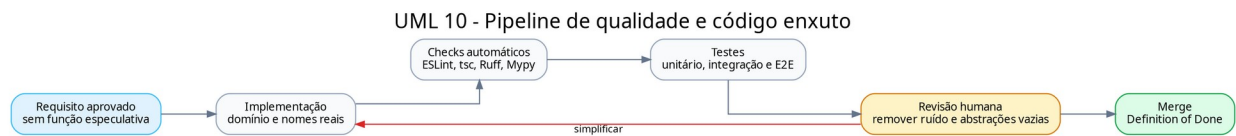


Figura 12 - Pipeline que impede boilerplate, ruído e código sem propósito.

9.1 Regras obrigatórias

Regra	Critério de aceite
Nenhuma linha sem requisito	Todo código aponta para caso de uso, bug, controle ou teste necessário.
Sem comentários óbvios	Comentários explicam decisão, risco ou contexto; não repetem a sintaxe.
Sem abstração prematura	Criar interface, factory ou helper somente com repetição real ou fronteira externa.
Nomes de domínio	Usar workOrder, receivable e stockMovement; evitar dataManager, processThing e utils genérico.
Tratamento de erro explícito	Proibido except genérico silencioso; erros de domínio e infraestrutura são distintos.
TypeScript estrito	Sem any fora de adaptadores justificados; DTOs e respostas validados.
Dependência mínima	Nova biblioteca exige problema, alternativas, impacto e decisão registrada.
Arquivos controlados	Alerta de revisão em arquivos acima de 300 linhas; não dividir artificialmente.
Complexidade controlada	Funções com múltiplos fluxos são simplificadas; meta de complexidade ciclomática até 10.
Sem código morto	Imports, flags, componentes, endpoints e testes não usados impedem merge.
Revisão de código assistido	Remover texto genérico, wrappers inúteis, fallbacks imaginários e testes que não validam comportamento.

9.2 Quality gates no CI

```
frontend: eslint && tsc --noEmit && vitest run && playwright test
backend:  ruff check && ruff format --check && mypy && pytest
security: dependency audit + secret scan + container scan
database: migrations up/down testadas em banco descartável
delivery: build das imagens + health checks
```

10. Dependências planejadas

Camada	Dependências essenciais	Política
ERP frontend	react, react-dom, typescript, vite, react-router, @tanstack/react-query, react-hook-form, zod, @hookform/resolvers, tailwindcss, lucide-react	Adicionar bibliotecas de gráfico e componentes somente no módulo que precisar.
Site institucional	astro, integração React opcional	Saída estática; sem servidor Node permanente.
API	fastapi, uvicorn, pydantic-settings, sqlalchemy, alembic, psycpg, argon2-cffi, PyJWT, python-multipart, google-genai, httpx	Usar biblioteca oficial Gemini; manter integração atrás de gateway.
Documentos	reportlab, qrcode, pillow, pymupdf	Reaproveitar apenas recursos comprovadamente necessários.
Testes e qualidade	pytest, mypy, ruff, vitest, testing-library, playwright, eslint	Executar localmente e no CI.
Infraestrutura	Docker, Docker Compose, PostgreSQL, Caddy	Redis e object storage ficam opcionais.

11. Organização do código

```
nexus/
  apps/
    web/                # ERP React
      src/app/
      src/features/auth/
      src/features/finance/
      src/features/commercial/
      src/features/purchasing/
      src/features/inventory/
      src/features/laboratory/
      src/features/assistant/
    site/                # institucional Astro
    api/
      app/core/
      app/auth/
      app/finance/
      app/commercial/
      app/purchasing/
      app/inventory/
      app/laboratory/
      app/assistant/
      app/reminders/
      app/audit/
      migrations/
  infra/caddy/
  tests/e2e/
  docker-compose.yml
```

Módulos importam contratos do core e serviços do próprio domínio. Não será permitido criar uma pasta “utils” como depósito genérico. Funções compartilhadas terão nome e responsabilidade claros.

12. API inicial por capacidade

Área	Endpoints principais
Autenticação	POST /auth/login, POST /auth/register, POST /auth/logout, POST /auth/recovery, GET /me
Administração	GET/POST /users, PATCH /users/{id}/status, PUT /users/{id}/roles, GET /audit
Lembretes	GET/POST /reminders, PATCH /reminders/{id}, POST /reminders/{id}/complete
Assistente	POST /ai/interactions, GET /ai/conversations/{id}, POST /ai/feedback
Financeiro	CRUD e relatórios de entradas, saídas, remanejamentos e previsão
Laboratório	O.S., status, diagnóstico, orçamento, recibo, anexos e histórico
Estoque	Itens, locais, saldos, movimentos, reservas e inventários
Compras	Solicitações, pedidos, status, recebimentos e anexos
Comercial	Clientes, equipamentos, contratos, venda, aluguel e patrimônio

13. Conceitos novos necessários para IA e lembretes

Conceito	Responsabilidade funcional
Conversation	Sessão de chat pertencente a usuário e empresa.
Message	Entrada, resposta e referência às fontes utilizadas.
Tool run	Ferramenta solicitada, parâmetros validados, status e duração.
AI usage	Modelo, tokens, latência e custo estimado para controle.
Reminder	Título, vencimento, fuso, vínculo operacional, status e dono.
Reminder delivery	Tentativa de entrega, canal, resultado e horário.
AI feedback	Avaliação do usuário e correção para evolução do produto.

Esses conceitos pertencem ao sistema novo. Eles não dependem das tabelas existentes e serão migrados somente quando houver equivalente útil no legado.

14. Docker local e VPS Hostinger

UML 11 - Paridade local e produção

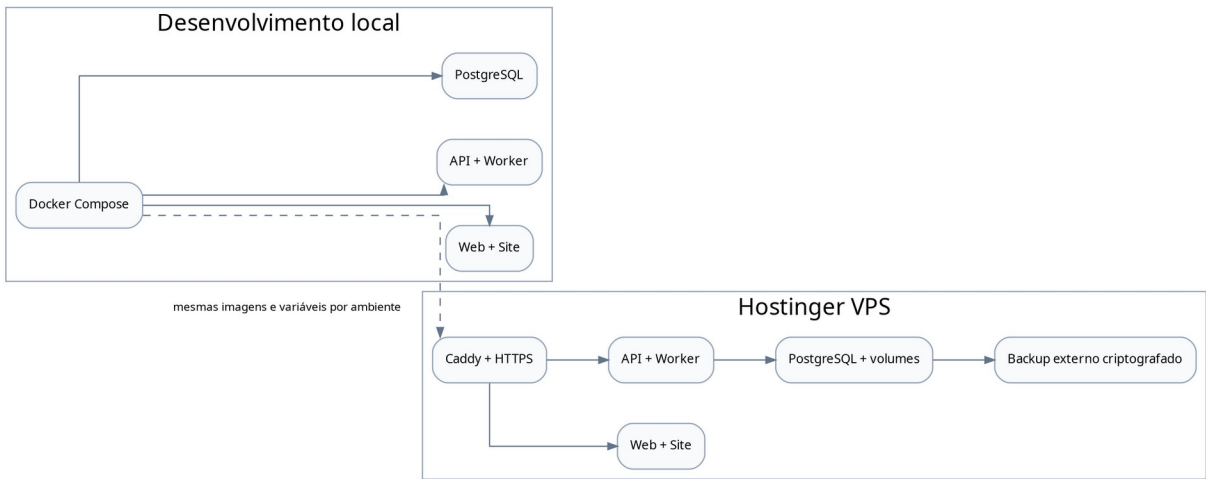


Figura 13 - Mesmo desenho de serviços no notebook e na VPS.

Serviço	Local	Produção Hostinger
site	Container de build + saída estática	Servido pelo Caddy em empresa.com.br
web	Vite dev server ou build	Servido pelo Caddy em app.empresa.com.br
api	FastAPI com reload	FastAPI sem reload, health check e limites
worker	Processo separado	Processo separado reiniciável
postgres	Volume Docker local	Volume persistente + backup externo
proxy	Caddy local opcional	Caddy obrigatório com TLS automático
gemini	Chave de desenvolvimento restrita	Secret de produção, orçamento e rate limits

Domínios previstos

empresa.com.br para o site institucional, app.empresa.com.br para o ERP e api.empresa.com.br apenas quando houver necessidade de exposição separada. O Caddy pode manter /api no mesmo domínio do ERP para reduzir complexidade de CORS.

15. Observabilidade, backup e recuperação

- Logs estruturados com request_id, usuário, empresa, rota, duração e status.
- Logs de IA separados: ferramenta, modelo, tokens, latência e resultado, sem registrar segredos.
- Health checks de web, API, banco, worker, espaço em disco e integração Gemini.
- Backup diário do PostgreSQL e arquivos, criptografado e copiado para destino externo.
- Teste mensal de restauração em ambiente isolado.
- Alertas de disco, falha de backup, fila de lembretes atrasada, erro 5xx e aumento anormal de custo da IA.
- Plano de rollback para release e para migration de banco.

16. Estratégia de testes

Nível	Cobertura mínima
Unitário	Regras de status, permissões, cálculo, validação de convite, parser de lembrete e políticas de IA.
Integração	API + PostgreSQL, uploads, transações, migrations e ferramentas do assistente.
Contrato	Schemas entre React, API e function calls Gemini.
E2E	Login, giro do card, cadastro pendente, aprovação, fluxos críticos dos módulos e lembrete completo.
Acessibilidade	Teclado, foco, labels, contraste, reduced motion e leitores de tela no card.
Segurança	RBAC, isolamento por empresa, brute force, upload malicioso, prompt injection e secret scan.
Carga	Consultas principais, PDFs, uploads e concorrência de lembretes.

17. Critérios de aceite específicos

ID	Critério
AC-LOGIN-01	Usuário válido entra sem recarregar toda a aplicação e recebe somente os módulos permitidos.
AC-LOGIN-02	Giro funciona por mouse, toque e teclado; em reduced motion não há rotação.
AC-REG-01	Cadastro sem convite não cria conta ativa.
AC-REG-02	Solicitante nunca escolhe perfil administrativo.
AC-AI-01	Nenhuma chave Gemini aparece no frontend, logs ou repositório.
AC-AI-02	Toda resposta baseada em dados internos apresenta links ou referências aos registros consultados.
AC-AI-03	IA não executa exclusão, pagamento, aprovação, estoque ou mudança de status autônoma.
AC-REM-01	Lembrete só é persistido após prévia e confirmação.
AC-REM-02	Worker não entrega o mesmo lembrete duas vezes mesmo com reinício.
AC-CODE-01	CI falha com lint, tipagem, teste, segredo, dependência vulnerável ou código morto detectado.
AC-DEPLOY-01	docker compose up inicia ambiente limpo com health checks aprovados.
AC-VPS-01	Reinício da VPS não perde banco, anexos ou lembretes pendentes.

18. Fases de implementação atualizadas

Fase	Entrega
0 - Especificação	Casos de uso, protótipos, permissões e Definition of Done.
1 - Fundação	Monorepo, Docker, React, Astro, FastAPI, PostgreSQL e Caddy.
2 - Identidade	Card reversível, login, convite, aprovação, sessão, RBAC e auditoria.
3 - Shell operacional	Menu, unidade ativa, notificações, anexos e componentes compartilhados.
4 - Financeiro	Operações, previsão, remanejamentos, balanço e auditoria.
5 - Laboratório	O.S., orçamento, recibos, fotos, PDFs, QR e status.
6 - Comercial/Estoque/Compras	Ciclos integrados e rastreáveis.
7 - Gemini e lembretes	Chat read-only, ferramentas, fontes, confirmações e worker.
8 - Site institucional	Conteúdo, SEO, contato e publicação.
9 - Migração e homologação	ETL dos dados legados, reconciliação, testes e entrada em produção.

19. Riscos e decisões preventivas

Risco	Prevenção
Cadastro aberto indevidamente	Convite ou aprovação obrigatória; role default sem acesso.
IA inventar ou misturar informação	Ferramentas internas, fontes, schemas estruturados e linguagem de incerteza.
IA causar alteração indevida	Sem acesso direto; confirmação e serviços com RBAC.
Custo imprevisível da API	Modelo configurável, rate limit, budget, cache de contexto quando útil e telemetria.
Código inflado por assistência de IA	Quality gates, revisão humana, limites de complexidade e remoção de boilerplate.
VPS única indisponível	Backup externo, monitoramento, restore testado e possibilidade de separar serviços depois.
Site público atingir ERP	Caddy, rotas isoladas, endpoint público mínimo e rate limit.
Acoplamento ao fornecedor de IA	Gateway próprio, ferramentas de domínio independentes e modelo configurável.

20. Referências técnicas oficiais

Gemini API - visão geral e Interactions API: <https://ai.google.dev/gemini-api/docs>

Function calling com Gemini: <https://ai.google.dev/gemini-api/docs/function-calling>

Structured outputs: <https://ai.google.dev/gemini-api/docs/structured-output>

Segurança de chaves da Gemini API: <https://ai.google.dev/gemini-api/docs/api-key>

Modelos Gemini: <https://ai.google.dev/gemini-api/docs/models>

Política de logs e compartilhamento: <https://ai.google.dev/gemini-api/docs/logs-policy>

Zero data retention em serviços elegíveis: <https://ai.google.dev/gemini-api/docs/zdr>

Rate limits: <https://ai.google.dev/gemini-api/docs/rate-limits>

Conclusão

O NEXUS será reconstruído como um sistema operacional confiável, não como uma reprodução do Streamlit. O card reversível melhora a entrada do usuário; a segurança continua no backend. O Gemini amplia consulta e organização, mas trabalha dentro de ferramentas, permissões, confirmações e auditoria. O padrão de código prioriza domínio, clareza e o menor número de linhas necessário para entregar comportamento testável.